Journal of King Saud University - Computer and Information Sciences 35 (2023) 101842

HOSTED BY

Contents lists available at ScienceDirect

Journal of King Saud University - Computer and Information Sciences

journal homepage: www.sciencedirect.com

Full length article

Autonomous solution for Controller Placement Problem of Software-Defined Networking using MuZero based intelligent agents

Ouafae Benoudifa, Abderrahim Ait Wakrime *, Redouane Benaini

Computer Science Department, Faculty of Sciences, Mohammed V University in Rabat, Morocco

ARTICLE INFO

Keywords:
Software-Defined Networking
Artificial Intelligence
Controller Placement Problem
Automated placement
MuZero

ABSTRACT

Controller placement is a critical issue in Software-Defined Networking, which has been identified as a potential approach to achieve more flexible network control and management. Deep Reinforcement Learning holds immense promise in achieving favorable outcomes through its ability to explore solution spaces and adapt to swiftly changing data streams. Furthermore, the advancement of reinforcement learning algorithms, like chess or Go, has inspired the concept of treating controller placement problem as a Markov game. These algorithms are employed to train intelligent agents capable of autonomously resolving this problem. This paper presents an intelligent system capable of optimizing the placement of controllers in Software-Defined Networking. The Muzero Reinforcement learning algorithm is used to train a model via self-competition which, by combining a tree search with a learned model, achieves superhuman performance in complex domains. Once trained, the model is integrated with an Software-Defined Networking controller, so that it is able to find the optimal placement of the game in a real network by incorporating performance metrics including latency, load, and overhead communication into the training process of intelligent agents. In order to show that our approach is feasible and efficient in practice, experiment results are provided as a benchmark.

1. Introduction

Software-Defined Networking (SDN) represents an innovative network architecture that incorporates various network technologies to construct flexible, scalable, agile, and easily manageable networks (Benzekki et al., 2016; Ahmad and Mir, 2021; Sarmiento et al., 2021; Singh and Jha, 2017; Tadros et al., 2020; Long et al., 2019). SDN introduces a novel perspective by decoupling the network into two distinct planes: the control plane and the data plane. This separation allows for centralized network control and enhances the efficiency of data transfer. Additionally, SDN brings programmability to networks, enabling the separation of the control plane and the data plane. It introduces a central maintenance mechanism and employs network devices solely responsible for forwarding data. This general-purpose equipment operates with a network operating system and fosters the development of new applications in lieu of traditional protocols. Notably, SDN introduces new protocols like OpenFlow and NETCONF. These advancements in SDN technology contribute to a reduction in capital expenditures (CAPEX) and operational costs, along with an increase in reliability and security. Moreover, SDN offers enhanced troubleshooting capabilities and ensures a fully controlled network environment.

The process of identifying appropriate locations for allocating controllers and effectively assigning switches to these controllers to achieve specific objectives is commonly referred to as the Controller Placement Problem (CPP) (Heller et al., 2012). One of the primary challenges that arise when deploying SDN in operational networks, particularly in large-scale environments, is the issue of system scalability (Singh and Jha, 2017; Tadros et al., 2020). While SDN offers significant advantages, it has its limitations in terms of scalability. In smaller networks, a single central controller can efficiently handle the manageable number of data flows. However, when scaling up to large operator networks, where millions of data flows need to be managed, relying solely on a centralized controller becomes a bottleneck that can hinder network performance (Hakiri et al., 2014). To address this scalability challenge

* Corresponding author.
E-mail address: abderrahim.aitwakrime@fsr.um5.ac.ma (A. Ait Wakrime).
Peer review under responsibility of King Saud University.

Production and hosting by Elsevier

https://doi.org/10.1016/j.jksuci.2023.101842
Received 8 August 2023; Received in revised form 24 October 2023; Accepted 8 November 2023
Available online 19 November 2023
1319-1578/© 2023 The Authors. Published by Elsevier B.V. on behalf of King Saud University. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

effectively, the use of multiple controllers is essential to effectively oversee network control across various domains (Killi and Rao, 2019). In the distributed architecture of SDN, network performance is inherently tied to the strategic placement of controllers and the resolution of the CPP. Therefore, determining the number of controllers required and their optimal placement within the network remains a critical consideration for achieving a fully distributed SDN controller architecture. The earliest studies on CPP were conducted by Sood et al. (2016). The performance evaluation of CPP relies on various metrics, including latency, load balancing, and communication overhead. The most frequently asked question concerning CPP (Controller Placement Problem) is: how many controllers are necessary, and where should they be strategically deployed within a multi-controller SDN architecture?

Numerous studies in this domain have attempted to consider various constraints known as metrics or optimized objectives. To determine optimal controller locations, these studies consider several factors, including: (1) Propagation delay between switches and controllers (2) Inter-controller latencies and the challenge of load balancing between controllers. (3) Identifying the optimal number and placement of controllers, which significantly impacts network performance. Recognizing that the CPP is considered NP-hard (Heller et al., 2012), with network partitioning emerging as the most widely adopted technique to address it. This technique involves dividing a wide-area network into subnets to minimize latency between controllers and switches, simplifying controller placement, and effectively managing load balancing and reliability. Various algorithms have been employed in these studies, including k-center, k-means, k-critical, as well as other clustering algorithms, linear and quadratic programming, bio-inspired algorithms, genetic algorithms, heuristic and greedy algorithms, simulated annealing algorithms, and machine learning algorithms. In contrast to previous works that considered only one or two metrics, our paper takes a different approach. We formulate the optimal controller placement as a multi-objective optimization problem with the following objectives: (i) Minimize the latency between controllers and switches. (ii) Minimize the overhead of communication among controllers (i.e., reduce the number of controllers). (iii) Ensure load balancing among controllers so that each controller handles a similar amount of traffic. Balancing these objectives can be challenging since some of them may contradict each other, such as objectives (i) and (ii). To address this, we employ the MuZero Reinforcement Learning (RL) algorithm to solve this complex optimization problem, seeking a fair trade-off among these objectives.

The main Contributions in our proposed approach are summarized as follows:

1. The design of a Markov game suitable to formulate the problem of controller placement in SDN Autonomous Optimization Controller Placement (AOCP) and provide a system model for it.
2. Then, the model has been trained using the MuZero model-based reinforcement learning algorithm

The rest of the paper is organized as follows: In Section 2, we provide an overview of existing approaches for CPP and offers background information on the subject. In Section 3, the paper discusses the relationship between SDN, Multi-Agent Systems (MAS), and RL problem within the context of game approaches. It also introduces the MuZero algorithm, which is a key focus of the study. In Section 4, the paper presents the proposed approach to address CPP issues in SDN, incorporating the MuZero algorithm as a central component. Section 5 details the implementation of the proposed approach and provides the results obtained from applying self-competition for leader node selection in a cluster. The paper concludes, in Section 6, by summarizing the findings of the study and discussing potential avenues for further research in this field.

2. Related works

2.1. Overview of existing approaches for CPP

To fulfill the requirements of optimized performance objectives, several algorithms or approaches have been proposed. These approaches can be categorized as follows:

2.1.1. Cluster-based CPP strategies

The authors utilized various methods, including density-based switch clustering (Liao et al., 2017), Clustering-based Network Partition (CNPA) (Wang et al., 2017), heuristics (Tanha et al., 2018), Fuzzy C-Means algorithm (Thalapala and Guravaiah, 2022), greedy approaches (Ishigaki et al., 2017), CPPKKF K-means, K-Harmonics means and firefly (Amin et al., 2021), as well as K-median, K-center (Sahoo et al., 2016), and K-medoids clustering algorithms (Lange et al., 2015a). The primary aim of these approaches is to minimize latency between each controller and its set of controlled switches, while considering factors such as load, resiliency, fault tolerance, and quality of service (QoS).

By employing density-based switch clustering, switches can be grouped based on their geographic proximity or communication traffic. This optimization minimizes latency by reducing communication delays between controllers and switches.

CNPA involves dividing the network into subnetworks or partitions, where each partition is associated with a specific controller. This allows for better load distribution among controllers and facilitates network management and control.

The Fuzzy C-Means algorithm, a clustering method that assigns fuzzy membership degrees to each switch for multiple potential clusters. This approach considers uncertainties and ambiguities in the data.

Greedy approaches seek to iteratively solve the problem by making locally optimal decisions at each step. While these approaches can be fast, they do not guarantee a globally optimal solution.

CPPKKF K-means, K-Harmonics means, firefly, K-median, K-center, and K-medoids are variations or extensions of the K-means algorithm, which is a commonly used clustering algorithm. These approaches aim to find K centers or medians that minimize the average distance between points and centers.

By utilizing these clustering approaches, control networks can be optimized in terms of latency, load, resiliency, fault tolerance, and QoS. This improves the overall performance of the network and provides higher-quality services to users.

2.1.2. Linear and quadratic programming CPP strategies

The authors introduced additional objectives of system cost and reliability and propose the utilization of queuing theory (Sridharan et al., 2017), linear programming (LP) (Sallahi and St-Hilaire, 2014), integer linear programming (ILP) (Müller et al., 2014; Savas et al., 2018; Ksentini et al., 2016), mixed integer linear programming (MILP) (Vizarreta et al., 2016; Qin et al., 2018), and mixed integer programming with quadratic terms in the constraints (MIQCP) algorithms (Auroux and Karl, 2014) to solve the system.

Queuing theory is a mathematical approach used to analyze and model the behavior of queues or waiting lines. By applying queuing theory, the authors can evaluate and optimize system performance metrics such as latency, throughput, and queue lengths. This allows for better resource allocation and management in the control plane.

LP is a mathematical optimization technique used to find the best outcome in a linear mathematical model. By formulating the CPP problem as a linear programming model, the authors can optimize the objectives of minimizing latency, load, and system cost while considering constraints such as the availability of resources.

ILP extends the linear programming approach by introducing integer variables, allowing for the optimization of discrete decision variables. In the context of CPP, ILP can be employed to determine the optimal assignment of controllers to switches, considering factors such as latency, load balancing, and reliability.

MILP combines both integer and continuous decision variables in a linear programming model. MILP can be used to optimize the placement and allocation of controllers in the network while considering multiple objectives such as minimizing latency, load, system cost, and ensuring fault tolerance.

Mixed integer programming with quadratic terms in the constraints (MIQCP) extends the MILP approach by incorporating quadratic terms in the constraints of the optimization problem. This allows for the consideration of non-linear relationships and interactions between variables. MIQCP can be utilized to optimize the CPP problem by considering non-linear factors such as the relationship between latency and load.

By employing these linear and quadratic programming CPP strategies, the authors aim to optimize the system by simultaneously considering multiple objectives, including latency, load, system cost, and reliability. This holistic approach helps in achieving an efficient and reliable control plane for the network

2.1.3. Bio-inspired CPP strategies

The authors consider various factors such as latency, load, cost, reliability, and resiliency in their proposed approaches for the optimal placement of SDN controllers. They employ different optimization algorithms including APSO (Accelerated Particle Swarm Optimization) (Wu et al., 2018), SSOA (Salp Swarm Optimization Algorithm) (Ateya et al., 2019), ABFO (Adaptive Bacterial Foraging Optimization) (Zhang et al., 2018), Firefly Algorithm (FFA) (Sahoo et al., 2018), Garter Snake Optimization algorithm (Torkamani-Azar and Jahanshahi, 2020), and VBO (Varna-based Optimization) (Singh et al., 2020).

APSO (Accelerated Particle Swarm Optimization is a variant of the Particle Swarm Optimization algorithm that aims to improve convergence speed and search efficiency. It simulates the behavior of a swarm of particles to explore the solution space and find the optimal controller placement that minimizes latency, load, cost, and enhances reliability.

SSOA is an optimization algorithm inspired by the collective movement of salps. It utilizes the concept of buoyancy and swimming behavior of salps to search for optimal solutions. The algorithm considers latency, load, cost, and reliability in determining the optimal placement of SDN controllers.

ABFO is a nature-inspired optimization algorithm based on the foraging behavior of bacteria. It mimics the bacterial foraging process to find optimal solutions. The algorithm adapts and adjusts its parameters dynamically during the search process to optimize the placement of controllers, considering latency, load, cost, reliability, and resiliency.

FFA is an optimization algorithm that models the flashing behavior of fireflies. It utilizes the attractiveness and intensity of light emitted by fireflies to guide the search for optimal solutions. FFA can be applied to find the optimal placement of SDN controllers, considering various objectives such as latency, load, cost, reliability, and resiliency.

Garter Snake Optimization algorithm is a metaheuristic algorithm inspired by the hunting behavior of garter snakes. It mimics the movement and hunting patterns of snakes to search for optimal solutions. The algorithm considers latency, load, cost, reliability, and resiliency in determining the optimal placement of SDN controllers.

VBO algorithm is an optimization technique that employs the concept of Varna, a classical Indian art form representing symmetry and harmony. VBO aims to find the optimal placement of SDN controllers by balancing the objectives of minimizing latency, load, cost, reliability, and resiliency, while maintaining a symmetrical and harmonious network structure.

By utilizing these optimization algorithms, the authors aim to find the optimal placement of SDN controllers that minimizes latency, optimizes load balancing, reduces cost, enhances reliability, and ensures network resiliency. These approaches contribute to improving the overall performance and efficiency of SDN-based networks.

2.1.4. Simulated annealing based CPP strategies

The authors propose simulated annealing-based strategies for the CPP. They utilize simulated annealing-based heuristic approaches (Aravind et al., 2022; Killi and Rao, 2018; Mohanty et al., 2020), as well as the Pareto Simulated Annealing (PSA) algorithm (Lange et al., 2015b), to optimize the placement of controllers in terms of load, latency, cost, and resiliency.

Simulated annealing is a metaheuristic algorithm inspired by the annealing process in metallurgy. It is used to search for global optimization solutions by simulating the cooling process of a material to reduce defects and reach a stable state. In the context of CPP, simulated annealing can be applied to find the optimal placement of controllers by iteratively exploring the search space and accepting occasional uphill moves to escape local optima.

The simulated annealing-based heuristic approaches utilized in these references aim to minimize the load among controllers, latency, and cost while maximizing resiliency. The heuristic algorithm iteratively evaluates candidate solutions and makes incremental changes to improve the objective function. By incorporating simulated annealing principles, the algorithm can effectively explore the solution space and escape local optima to find better solutions.

The PSA algorithm extends simulated annealing to handle multiple conflicting objectives. It applies the concept of Pareto dominance, which compares solutions based on their trade-off between different objectives. PSA explores the search space to identify a set of Pareto-optimal solutions that represent the best compromise between load, latency, cost, and resiliency.

By using simulated annealing-based strategies, the authors aim to find optimal controller placements that achieve a balance between minimizing load, latency, and cost, while maximizing resiliency. These approaches enable the optimization of the control plane in software-defined networks, leading to improved network performance, resource utilization, and robustness.

2.1.5. Genetic algorithm based CPP strategies

Several Genetic Algorithm (GA) based strategies are proposed for solving the CPP. These approaches utilize Genetic Algorithm (Yuan et al., 2018; Ahmadi and Khorramizadeh, 2018; Hu et al., 2016), Cooperative Genetic Algorithm (CPGA) (Jalili et al., 2019) and Gradient-Descent-based (GD) algorithm (Huang et al., 2020) to address the optimization challenges in CPP.

Genetic Algorithm (GA) is a metaheuristic algorithm inspired by the process of natural selection and evolution. It mimics the principles of genetics, such as crossover, mutation, and selection, to iteratively search for optimal solutions. In the context of CPP, GA can be employed to find the optimal placement of controllers by representing potential solutions as chromosomes, and using genetic operators to generate new candidate solutions.

Cooperative Genetic Algorithm (CPGA) is an extension of GA that incorporates cooperative behavior among multiple genetic algorithms. CPGA employs multiple GA instances that cooperate by sharing information and exchanging solutions to enhance the search process. This collaboration helps to diversify the search space and improve the quality of the solutions for CPP.

Gradient-Descent-based (GD) algorithm is a search algorithm that utilizes gradient information to iteratively improve the objective function. In the context of CPP, GD algorithm aims to minimize latency, load, and cost by iteratively adjusting the placement of controllers based on the gradient of the objective function. This approach can converge towards a local optimum but may not guarantee finding the global optimum.

By employing these Genetic Algorithm-based strategies, the authors aim to find optimal solutions for CPP by considering objectives such as latency, load, cost, and resiliency. These approaches enable the exploration of the solution space and the discovery of near-optimal or optimal placements of controllers in software-defined networks.

2.1.6. Others CPP strategies/machine learning

The authors propose several other strategies for CPP that consider latency, load, and reliability. These approaches incorporate a combination of different algorithms, including Hybrid Differential Evolution and Whale Optimization (DEWO) algorithm (Keshari et al., 2021), Spectral clustering algorithms Heuristic ant colony algorithms simulated annealing algorithm (Li et al., 2022), Machine Learning (ML) (He et al., 2017; Mostafaei et al., 2018), Wisdom of Artificial Crowds based genetic algorithm (Thalapala et al., 2022), and DDCP (Deep Q-Network based Dynamic Clustering and Placement) heuristic algorithm (Bouzidi et al., 2022).

The Hybrid Differential Evolution and Whale Optimization (DEWO) algorithm combines the concepts of Differential Evolution (DE) and Whale Optimization Algorithm (WOA). DEWO aims to find optimal controller placements by iteratively improving solutions through evolutionary operators and simulating the behavior of whales. This approach considers latency, load, and reliability in the optimization process.

Spectral clustering algorithms are graph-based methods that partition the network into clusters based on spectral properties of the adjacency matrix. By applying spectral clustering, the authors aim to identify optimal clusters of switches and controllers that minimize latency, load, and enhance reliability.

Heuristic ant colony algorithms simulate the foraging behavior of ants to solve optimization problems. In the context of CPP, ant colony algorithms aim to find the optimal placement of controllers by iteratively exploring the solution space and adjusting the placement based on pheromone trails. This approach considers latency, load, and reliability to guide the search process.

The simulated annealing algorithm, as mentioned earlier, is also utilized in this context. It employs the principle of simulated annealing to iteratively search for optimal placements of controllers by considering latency, load, and reliability.

ML approaches utilize algorithms and techniques to learn patterns and make predictions or decisions. In the context of CPP, ML can be applied to model the relationship between controller placement and performance metrics, such as latency, load, and reliability. ML algorithms can learn from historical data and make informed decisions to optimize the placement of controllers.

The Wisdom of Artificial Crowds based genetic algorithm leverages the collective intelligence of a population to solve optimization problems. It combines genetic algorithms with the idea of collective decision-making to find optimal controller placements considering latency, load, and reliability.

DDCP heuristic algorithm incorporates Deep Q-Network (DQN), a reinforcement learning technique, to dynamically cluster switches and controllers and optimize their placement. By considering latency, load, and reliability, DDCP aims to find near-optimal solutions through a heuristic approach.

By using these various strategies, including hybrid algorithms, clustering methods, heuristics, simulated annealing, machine learning, and reinforcement learning, the authors aim to optimize controller placement in terms of latency, load, and reliability. These approaches contribute to enhancing the performance and efficiency of SDN.

Previous research efforts have extensively explored various aspects of resolving controller placement problem in SDN environments, focusing on key performance metrics such as latency, load, cost, reliability, fault tolerance and QoS. Notably, in Clustered approach all papers have addressed the latency aspect, while in paper (Liao et al., 2017) fault tolerance was defined as optimized objective. Meanwhile, paper (Tanha et al., 2018) added QoS Resiliency as optimized objective. In Linear and quadratic programming approach most papers focused on latency optimization while paper (Sridharan et al., 2017) addressed resiliency and paper (Sallahi and St-Hilaire, 2014)addressed cost optimization. Meanwhile papers (Müller et al., 2014; Auroux and Karl, 2014) added load and reliability. In Bio-inspired approach all papers addressed latency optimization, while papers (Wu et al., 2018; Ateya et al., 2019; Zhang et al., 2018; Singh et al., 2020) focused on load and papers (Wu et al., 2018; Zhang et al., 2018) on reliability, but papers (Ateya et al., 2019; Zhang et al., 2018) addressed cost optimization. In Simulated annealing approach all papers addressed latency optimization, while paper (Mohanty et al., 2020) focused on both load and resiliency. Meanwhile in paper (Lange et al., 2015b) cost was defined as optimized objective. In Genetic algorithm approach all papers addressed both latency and load optimization, while in paper (Hu et al., 2016) reliability was the optimized objective. In Others approach all papers addressed latency optimization, while in papers (Li et al., 2022; Bouzidi et al., 2022) load was the optimized objective. Meanwhile paper (Thalapala et al., 2022) added reliability.

2.2. Comparative analysis of existing approaches for CPP

Table 1 presents a classification of different approaches based on their optimized objectives in the context of control placement problem. The approaches are evaluated in terms of various criteria such as latency, load, cost, reliability, resiliency, fault tolerance, and quality of service (QoS). Each approach is assigned a corresponding symbol (the "+" symbol indicates that the corresponding approach utilizes and optimizes the criterion) for each criterion. Despite of contributions of different approaches cited before, there are some Complementarity of the algorithms used. For example, in greedy and heuristic algorithms they use in their process in solving the CPP: An empty set, then Iterate all the nodes and rank them according to the metric by selecting the locations that optimize the goals. Among the limits of these algorithms we can mention: The number of required controllers is already given, and for each location scenario they list all the nodes. Partitioning into subnets was the first solution proposed and heuristic algorithms are applied to each cluster or domain. In heuristic and metaheuristic approximations, they are based on paretooptimal whose principle is to return a solution called pareto-optimal front which verifies the condition of not having another solution which improves the solution found when all the objectives are taken into account. Among the algorithms used are: greedy, genetic, PSO (Particle Swarm Optimization) and annealing algorithms. In Clustering, its principle is to divide the network into sub-networks and implement the objectives in each so that the convergence of objectives is achieved in the global network. The algorithms used are: K-means algorithm for clustering is the most used but with its limits, other derivatives had been proposed such as k-means++, k-harmonic means, Firefly, k-medians and K-medoids and k-center. The clustering approach is the most used in CPP but its realization becomes difficult when it comes to the repetition of the partitioning process when the traffic is dynamic. In general in the SDN the performance depends on the controllers and the switches, the first objective which was optimized was to take into consideration the impact of the latency with all its derivatives like the maximum latency, worst latency Switch-Controller and average latency switch- and controller and controller-controller, but these metrics remain insufficient that is why other metrics have been proposed such as load balancing, reliability, energy saving, controller capacity and optimization of their number, fault tolerance, cost, resilience and QoS. In order to have an efficient SDN architecture, the integration of all these constraints while not forgetting their interdependence will be a main goal. Due to the conflict between these different metrics, multi-objective optimization was proposed in several studies. But the fact that the solutions are based on algorithms and each of these algorithms solves a specific problem and the combination of several objectives remains a challenge so we must think about proposing a more efficient, a solution that will avoid performance degradation by reducing manual intervention such as manual reconfiguration, installation of new equipment and maintenance. This is why we can think of using artificial intelligence (AI) as a solution for self-solving of the CPP by exploiting its advanced features.

Table 1
Classification by approaches and optimized objectives.

Approach	Latency	Load	Cost	Reliability	Resiliency	Fault tolerance	QoS
- Clustered (Section 2.1.1)	Liao et al. (2017), Wang et al. (2017) Thalapala and Guravaiah (2022) to Lange et al. (2015a)	Lange et al. (2015a)	–	–	Tanha et al. (2018)	Liao et al. (2017)	Tanha et al. (2018)
- Linear and quadratic programming (Section 2.1.2)	Savas et al. (2018) to Auroux and Karl (2014)	Müller et al. (2014) to Auroux and Karl (2014)	Sallahi and St-Hilaire (2014)	Müller et al. (2014)	Sridharan et al. (2017) Müller et al. (2014) to Savas et al. (2018)	–	–
- Bio-inspired (Section 2.1.3)	Wu et al. (2018) to Singh et al. (2020)	Wu et al. (2018) to Zhang et al. (2018) Singh et al. (2020)	Ateya et al. (2019), Zhang et al. (2018)	Wu et al. (2018), Zhang et al. (2018)	–	–	–
- Simulated annealing (Section 2.1.4)	Aravind et al. (2022) to Lange et al. (2015b)	Singh et al. (2020), Mohanty et al. (2020)	Lange et al. (2015b)	Lange et al. (2015b)	Mohanty et al. (2020)	–	–
- Genetic algorithm (Section 2.1.5)	Yuan et al. (2018) to Huang et al. (2020)	Yuan et al. (2018) to Huang et al. (2020)	–	Hu et al. (2016)	–	–	–
- Others (Section 2.1.6)	Keshari et al. (2021) to Bouzidi et al. (2022)	Li et al. (2022), Bouzidi et al. (2022)	–	Thalapala et al. (2022)	–	–	–

3. Background

3.1. SDN

SDN helps the network administrator to respond rapidly to network requirements by using centralized controllers. SDN has changed the traditional network limitations by decoupling the data plane from the control plane to forward network traffic. SDN is designed to effectively provide connectivity and management to deal with complexity and heterogeneity in the traditional network forwarding unit and inefficient network management (Xia et al., 2014; Farhady et al., 2015). With the rapid development of internet services, SDN is now widely used in large mobile network (Pujolle, 2015). SDN provides many Characteristics, including programmability, centralized control, error-prone configuration, complex network control, network flexibility, improved performance, easy implementation, efficient configuration and enhanced management (Benzekki et al., 2016). Controller placement is a critical issue in SDN in large networks, where the traditional centralized control model faces scalability challenges as the network grows. To address this, our proposed solution leverages the benefits of SDN, such as centralized control and network flexibility, to create a more adaptable network environment.

3.2. MAS and the RL

A MAS consists of multiple agents interacting with each other in a common environment. The concept of an agent has recently become important in AI. The MAS can solve problems that are difficult or impossible for an individual agent. Intelligence may include methodical, functional, procedural approaches, algorithmic search or RL. The RL combined three axes between optimal control, animal psychology and AI. RL has known a huge success by the appearance of alphago algorithm (Silver et al., 2016) which leaded to solve complex computational tasks. RL shows in general how an agent should learn to make decisions, it is based on trial and error learning (Kaelbling et al., 1996; Arulkumaran et al., 2017). The main elements of the RL models are shown in Fig. 1. At each time-step, an agent with a state si in an environment perform an action, this action triggers a response from the environment in which the environment rewards the agent for taking this action and the agent move to a new state si+1. The main goal of the agent is to maximize the reward. This process becomes more complex in dynamic environment where the information becomes partial. Those

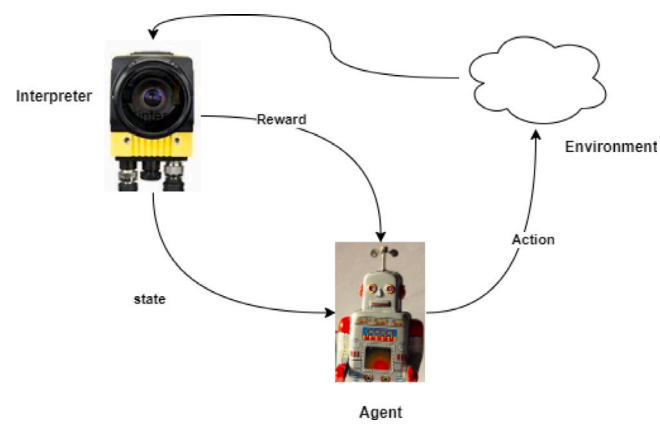

Fig. 1. The main elements of reinforcement learning scenario.

processes studied in RL are Markov Decision Processes (MDP) which are defined by a six-component tuple as shown in (1) where: (S, A, Psi+1si, Rasi+1si) (1) S: is a set of states called the state space A: is a set of actions called the action space Psi+1si: is the probability that action a in state si at time t will lead to state si+1 at time t+1 Rasi+1si: is the immediate reward received after transitioning from state si to state si+1, due to action a. Reinforcement learning algorithms are categorized into one of two categories — model-free reinforcement learning (MFRL) or model-based reinforcement learning (MBRL). In the first one an agent tries to find the optimal policy without having a model of the environment as the Qlearning (QRLref) algorithm. While in the second one the agent uses its interactions with the environment to learn about the environment's dynamics, and thereby model it. The algorithm Muzero is one of the most recent example of MBRL. MAS and RL concepts, as mentioned earlier, play a crucial role in our solution. They allow us to dynamically model the network environment and make informed decisions regarding controller placement. This integration empowers our system to adapt to changing network conditions and achieve optimal results.

3.3. Games theory

Game theory is involved in multi-player environments to resolve the problem of selecting optimal actions among rational agents (Bowling

and Veloso, 2000). A game is a mathematical object, which describes agent's interaction with the environment simultaneously or in turn in order to obtain a payoff (Nowé et al., 2012). The game theory is the most technical domain used by reinforcement learning. It has been used to examine multi-agent interactions (Tuyls et al., 2018) Games are classified by the following properties: (Browne et al., 2012) Zero-sum: The originally game whether the sum of all players' rewards is 0 or not. In the competition between two players the rewards sum to 0. Information: Whether the state of the game is fully or partially observable to the players Determinism: Whether the result of the game relies on chance factors Sequential: Whether actions are applied sequentially or simultaneously between agents Discrete: Whether actions are applied in real-time or discrete. Game theory principles are also at play in our solution. By framing the controller placement problem as a strategic interaction between network elements, we apply game theory to optimize the placement of controllers. This approach enables us to consider various network scenarios and dynamics, ensuring that the placement strategy is adaptable and efficient.

3.4. MuZero algorithm

MuZero's ability to develop a model from its own environment and use it to successfully plan its actions demonstrates a significant advance in reinforcement learning (Schrittwieser et al., 2020). The Intelligence of agents relies on skill and adaptivity of an agent, and the agent should be tested on a wide range of environments. In other side the algorithm which excels on broad intelligence will be chosen then other ones, Muzero is an algorithm realizing previous criteria (St John, 2020). The common factor between IA algorithms like Q-learning and AlphaZero is that they learned before the rules of games that they should master, in the contrast Muzero does not need to be provided by the rules to be able to master them. The main approach used by Muzero is looking forward, by using this approach the algorithm sees all possibilities for future cases to plan some steps before acting. In some way the Muzero combines the better in the approaches looking forward and modeling, instead of modeling everything it will look in important factors to make decision like what humans do. Muzero, when he has to make decision takes into account three factors, the first one is the result of its previous decisions, the second one its current case and the last one the best course for next steps. Incorporating reinforcement learning, specifically the MuZero algorithm, is a key component of our approach. MuZero's ability to learn and plan actions in complex environments aligns perfectly with the dynamic nature of SDN. By training a model via self-competition, we harness MuZero's capabilities to tackle the controller placement problem with superhuman performance. The model, once trained, becomes an integral part of an SDN controller, enabling it to find the optimal controller placement in real network scenarios.

4. System model and problem formulation

The proposed approach for controller placement in the SDN topology as shown in Fig. 2 is based on a self-competition game using the Muzero algorithm. The main objective of this approach is to determine the optimal locations for placing controllers within the network, taking into consideration both latency reduction for switch-controller communication and controller-controller communication overhead. To begin, the SDN network is represented as a graph, where nodes can function as switches or controllers. The proposed approach for controller placement in the SDN topology incorporates a clustering strategy to further enhance the decision-making process. Clustering is used to group network elements.

The self-competition game with the Muzero algorithm allows each controller to make intelligent decisions regarding its placement. By simulating a competition against itself, the controller can explore different placement strategies and evaluate their impact on the network performance. During this process, the controller considers multiple criteria, including communication latency and load balancing.

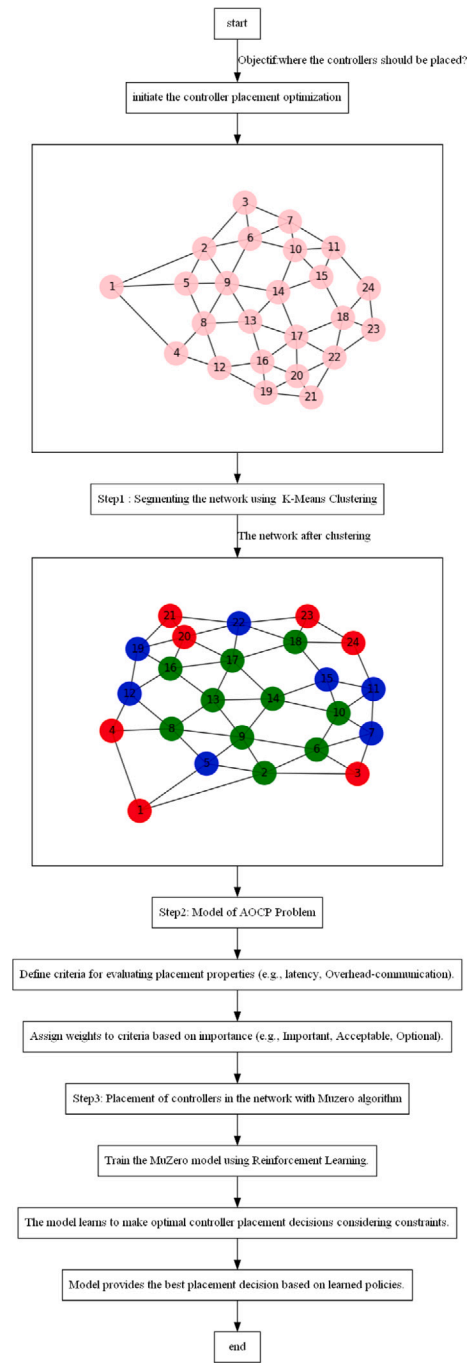

Fig. 2. Autonomous Optimization Placement controller (AOCP).

4.1. Step 1: Problem description

In this section, we describe the procedure carried out to place the controllers in the clusters of the SDN topology. Fig. 3 represents an illustrative example of controller's selection in SDN, as we can observe on this Figure the network is composed of nodes network. These nodes can have covered a large geographic zone. Each node has a position in the network and can be either a Switch or a Controller. The placement of controllers can lead to two scenarios: the first one is that the selection of controllers close to the switches leads to reduce the latency between switches and controllers (i.e, S-C communication overhead), however it leads to create a long path among controllers. Meanwhile, the reduce of distance among controllers (i.e, C–C communication overhead) leads

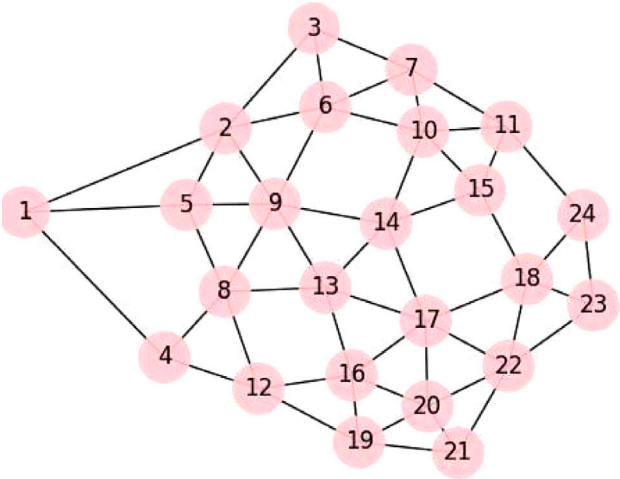

Fig. 3. Example of a placement controller problem in a SDN.

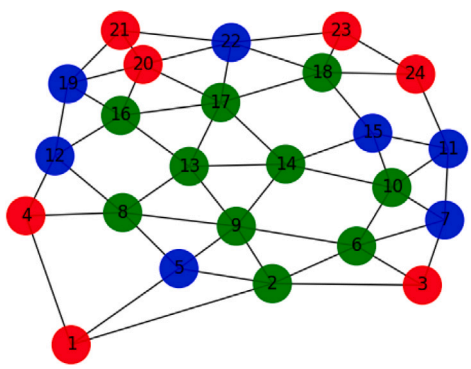

Fig. 4. Placement of controllers in the topology with clustering.

to create path with high latency between switches and controllers. The main objective is to find which the controller placement is selected based on a criteria set like latency and load balancing. So The optimum placement controller will be the one which satisfy the two objectives, but the problem is that the two objectives are contradictory, for that we will use self-competition game using Muzero algorithm to decide where to place the controller when it is looking for placement by making decision based on criteria.

Step 1 is intended to segment the network using K-Means Clustering examining the placement of controllers in relation to the whole network is not an efficient technique, since applying the game to the entire network is a difficult operation. As a result, segmenting the network into clusters improves the efficiency with which switches are assigned to the controller. In this work we used K-means algorithm for clustering. Fig. 4 shows the topology and the switches in each cluster; e.g., in the blue cluster, we can observe that there three switches. In this study, our objective is to select the optimum placement for a controller in a cluster. For example, in blue cluster i.e., S5, S7, S11, S12, S15, S19 and S22, the place will be selected through Muzero algorithm. Similarly, other numbers with nodes such as S1, S3, S4, S20, S21, S23 and S24 denotes another cluster, where these switches placement will be chosen through the Muzero algorithm.

The kmeans algoritm has two main parameters the center (the initial node which are selected to perform clustering algo) and the centroid (the actual center of each cluster which is eventually found by clustering algo). Firstly, the algo initialize K clusters and allocate one center for each cluster using random sampling, secondly it allocates nodes into one of the clusters based on Euclidean distance, then recalculates

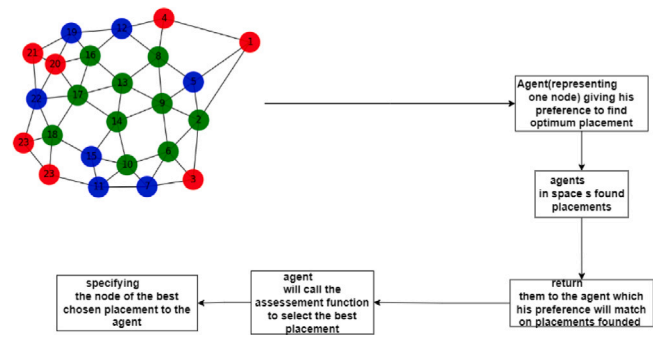

Fig. 5. Placement of controllers in the network with AOCP model.

centroid for each cluster and repeat the second and third steps until there is no change in each cluster.

4.2. Step 2: Model of AOCP problem

We formulate the AOCP problem as follows:

$$\begin{aligned}\text{AOCP} = (&N = \{d_i \mid i \in N\},\\ &C = \{C_i \mid i \in N\},\\ &G = (N, R, O),\\ &T = \{\tau_0, \tau_1, \ldots, \tau_{\text{fin}}\})\end{aligned}$$

where N defines a dynamic set of request nodes (switches and controllers) available in a specific time interval. The set C represents autonomous controllers that can only communicate within a limited distance. G is a graph with a set of nodes N and a set of links (routes) R, where $R_{ij} \in R$ represents the link between nodes i and j. The function o is an evaluation function associated with each link $r \in R$, providing a value θ_r that represents the measurement of distance for calculating latency between controllers and switches. T represents the temporal space of the problem.

Definition 1. An autonomous controller is characterized by its capacity $c_a : C \to \mathbb{N}^+$, its latency calculated as a function of distance $l_{fd} : C \to \mathbb{R}^+$, and a limited communication distance $d_{ng} : C \to \mathbb{R}^+$. Thus, $C = (c_a, l_{fd}, d_{ng})$. The initial place of the controller is denoted as $p_i : C \times T \to N \cup R$, and its optimal place is denoted as $p_o : C \times T \to N$, both of which depend on time.

Definition 2. two nodes can either be linked directly or no. Indeed, the fact of being limited in time and in distance communications, this can create connected groups.

Definition 3. having the process valid for any controller. Then, multiple controllers can have the same requests from other nodes. also the Communication distance will depend on the bandwidth of the links.

The optimality of the controllers' process is defined by its objective function, but in order to assess the feasibility of this placement process. There is a need to measure functional indicators that are as follows: QoS: Percentage of valid requests out of all requests launched. Validity: is the terminal validation of placements controllers calculated from the placements between initial placement and optimized placement. Overhead-communication: Total number of communications exchanged during the placement process which will allow to launch if there is a communication overload.

The criteria form an evaluation function for a property. The different properties are defined on the Table 2 that includes examples of preference criteria on placements properties. Each controller forms a set of criteria on the properties of the placement spaces and a criterion is 1

Table 2
Example of preference criteria on properties of placement.

Properties	Threshold	Description
Distance	Dmax	Distance between the node and final placement
Latency	lmax	Latency between the node and others nodes
Overhead communication	Mcmax	Max node–node overhead communication which can be tolerated in the network

if the criterion is met and 0 if not. Minimum latency is a property that the controller can put a criterion to specify its placement preference. Latency is a property which the node can put a criterion to specify its preference for latency. In Fig. 5 the agent sends a request to agents that are in an s space. Agents find placements with all properties in space s and returns them to the agent Who will match his preferences on the different placements found. This agent makes the assessment to select a placement who best meets these criteria and responds to the agent by specifying the node of the best chosen placement. The weighting of the criteria by a level of importance will be used whose values are. Important, acceptable, optional. For example, if a criterion on the latency property has an importance (Important), this property should have priority on the others less important. The criteria can have the same importance. The placement which satisfied the most of criteria with the max importance will be chosen. The criteria represent the eval function for a metric (latency, overhead communication. . . .)

The equation that assesses a placement p with respect to the list of criteria that correspond to the controller's preferences is eval(P). It returns a numerical value equal to the sum of the satisfactory criteria weighted by their importance. Here, Cri(p) is 1 if the criterion Cri is satisfied by the place P, and $N_{\text{cr}}^{\text{cri.imp}}$ represents the number of criteria N_{cr} raised to the power of cri.imp, which indicates the importance of the criterion. The equation can be written as:

$$\text{Eval}(p) = \sum_{\text{cri}\in\text{criteria}} \text{Cri}(P) \cdot N_{\text{cr}}^{\text{cri.imp}} \tag{1}$$

The process determines a placement that better matches the controller's preferences. A placement list is sent to this agent which will first have the first default placement as an optimized placement found then going through all the placements in the list by evaluating each time a placement of the list with the optimized placement that is already found using the function (a). If a better p placement is found, that placement will become the best one.

4.3. Step 3: Model system using MuZero algorithm

After studying different approaches and techniques used by different searchers who tries to find the efficient solution for problem placement Controller in SDN, we present our approach which lead to have an intelligent placement Controller by integrating AI. CPP is considered like a planning and sequential making decision problem to achieve the optimal location. In multiple controllers' placement, many constraints affect the SDN performance such as reliability, controller load balancing, latency, operational costs, and response time of events. So in the most the formulation is hybrid metrics or multi-objective optimization problems about controller location or placement. RL is a framework of sequential making decision which had a lot of success in resolving planning problems. In this work we propose to use the RL MuZero algorithm to train a model that is able to decide which placement controller is optimal with imposed constraints mentioned before.

4.3.1. Constrained Markov decision process

The optimal placement is measured as maximal preference. The placement of controllers can be formulated as constrained MDP (Markov Decision Process) CMDP tuple <S, A, P, Rpreference, γ > where S is the set of states, a list of placement at the first time which will represent the default and optimal placement found then ; A is the set of available state-dependent actions which will be browsing all the placements in the list by evaluating each time a placement in the list with the optimized placement that is already found using the function(a) if a better p placement is found, it will become the best placement; R : S × A → R is a bounded reward function, it is calculated using function (a). In this work we have chosen to reward only the winner of the game at the end of it, giving a null reward to the rest of the actions. It must be taken into account that the game also ends after a certain number of moves, thus avoiding infinite games. Games that end by this criterion are considered a draw and neither player receives a reward. In order to train a controller capable of finding his optimal placement, the rewards shown in (b) have been implemented.

Les équations sont données par :

$$r = \begin{cases} \max(\text{eval(p)}) & \text{if winner=optimal placement} \\ 0 & \text{otherwise} \end{cases} \tag{2}$$

P : S × A × S → [0, 1] is the transition function, where P (s' |s, a) is the probability of transitioning to state s' (next placement) from s when action a was taken; $\gamma \in [0, 1]$ is the discount factor. A policy π: S × A → [0, 1] is a mapping from a state to a distribution over actions where π (a|s) denotes the probability of taking action a in state s. The goal is to obtain a policy which maximizes the expected sum of discounted rewards J_π^R

$$\max_\pi (J_\pi^R)(s) := \mathbb{E}_{a\sim\pi,\, s\sim P}\left[\sum_t y^t R(s_t, a_t)\right] \tag{3}$$

A Constrained Markov Decision Process (CMDP) (Altman, 1999) extends the MDP formulation by introducing k constraint functions $c_k : S \times A \to \mathbb{R}$ and corresponding thresholds β_k. The goal is to obtain a policy which maximizes the expected sum of discounted rewards subject to the discounted constraints ($J_\pi^{(c_k)}$) being under the target thresholds β_k.

$$\max_\pi (J_\pi^R) \quad \text{s.t.} \quad J_\pi^{(c_k)} \le \beta_k \quad \forall k \tag{4}$$

In our case, the controller placement constrained objective to optimize is defined as:

$$\max_{\pi(placement_t)} (J_\pi^{\text{Preference}}) \tag{5}$$

4.3.2. MuZero

A new Reinforcement Learning algorithm (Schrittwieser et al., 2020)based on deep neural networks and Monte-Carlo tree search (MCTS) planning algorithm (Coulom, 2006) to model-based RL that achieves state-of-the-art performance in Atari 2600, a visually complex set of domains, while maintaining superhuman performance in precision planning tasks in various domains like Chess, Go, Shogi, and Atari. MuZero builds upon powerful search and search-based policy iteration algorithms, and incorporates a learned model into the training procedure. Unlike its predecessors, MuZero does not need to interact with the environment for planning; instead it uses a learned model of environment dynamics to look ahead.

4.3.3. Constrained controller placement with MuZero

In Fig. 6, the game is set up as a self-competition match between two players, the node and itself, in which the node tries to be identified as the controller in a network. Although the MuZero algorithm was designed to deal with board games or video games, in this case, the environment to be simulated is a network of interconnected nodes.

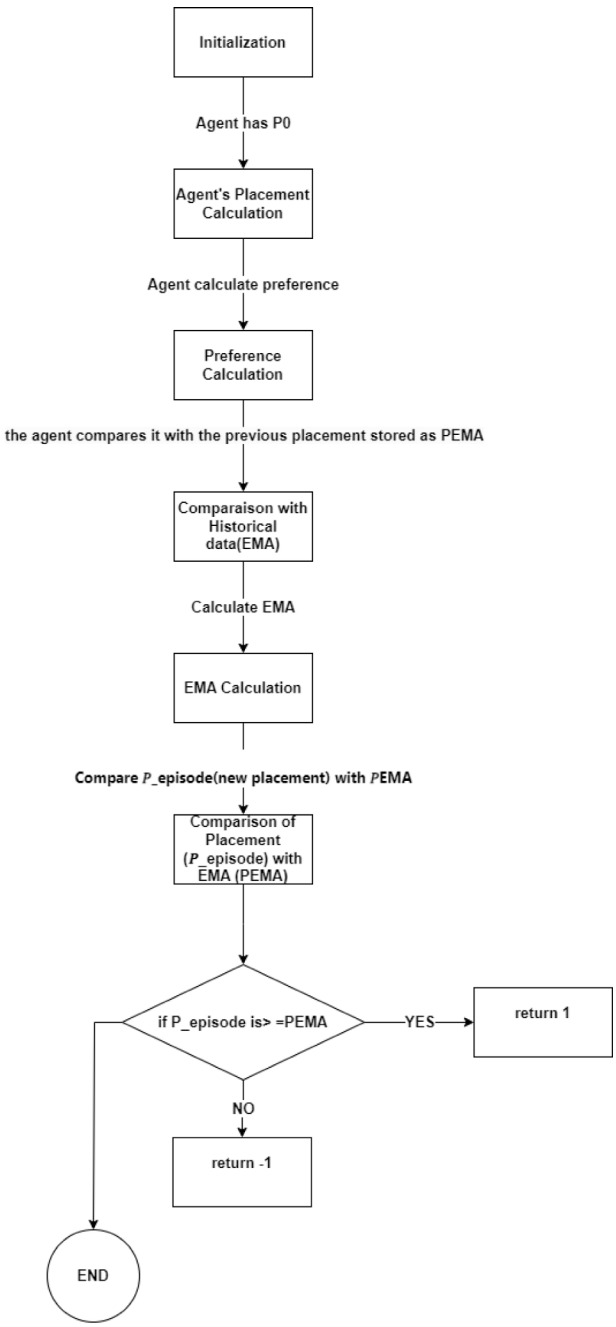

Fig. 6. Placement of controllers in the network with MuZero algorithm.

When the agent has a placement P_0, it calculates its preference and compares it with the previous placement, which will be recovered each time by using:

$$\text{EMA} = \left[\text{Price(today)} \times K\right] + \text{EMA(yesterday)} \times (1 - K)$$

where $K = \frac{2}{N+1}$ and N is the number of days in EMA.

The exponential moving averages are used to track preferences in relation to the historical average during the training course in a buffer. Then, the agent compares the P_{episode} (new placement) with P_{EMA}. The return of the episode is set to ± 1 depending on whether the episode is better than EMA, and the new P_{EMA} becomes P_{episode} (Update).

The pseudo-code, like Algorithm 2, Algorithm 3 and Algorithm 4, used in Algorithm 1 shows how a self-competition is executed.

$$\text{Return} = \text{sgn}(P_{\text{episode}} - P_{\text{EMA}}) \tag{6}$$

Algorithm 1: controller placement with self-competition

Input: Dataset D of placements with their preferences
Input: Buffer $B \leftarrow [Placement_i] \rightarrow (preference_0, 0)\ \ \forall Placements$
// Buffer B containing placement preferences
Input: α Buffer update factor
Output: Selected placement based on self-competition.
1 **while** $(Placement_i == none)$ **do**
2 $Placement_i \sim D$;
3 Preference_episode ← PLACE_CONTROLLER($Placement_i$);
4 UPDATE_BUFFER(B, Preference_episode, α);
5 $Placement_i \leftarrow$ SELF_COMPETITION_REWARD(B, $Placement_i$, Preference_episode)
6 **return** $Placement_i$

Algorithm 2: UPDATE_BUFFER Procedure

Input: Buffer B
Input: $Placement_i$
Input: Preference_episode
Output: the modified or updated Buffer B itself, which has been modified with the new Preference_episode value.
1 $B[Placement_i][0] = (1-\alpha) \cdot B[Placement_i][0] + \alpha \cdot$ Preference_episode;
2 **return** B

Algorithm 3: SELF_COMPETITION_REWARD Procedure

Input: Buffer B
Input: $Placement_i$
Input: Preference_episode
1 $PreferenceEMA \leftarrow B[Placement_i]$;
2 **if** $Preference_episode \geq PreferenceEMA$ **then**
3 **return** 1
4 **else**
5 **return** -1

Algorithm 4: PLACE_CONTROLLER Procedure

Input: criteria_1 $Cr1$
Input: criteria_2 $Cr2$
Input: weight_criteria_1 $w1$
Input: weight_criteria_2 $w2$
Input: $Placement_i$
Output: Preference_episode
1 Preference_episode $\leftarrow w1 \cdot Cr1 + w2 \cdot Cr2$;
2 **return** Preference_episode

In Algorithm 1 (lines 1 to 6) describes the main procedure for selecting controller placements using a self-competition approach. It takes a dataset D of placements with their preferences as input and iteratively selects placements while updating a buffer B to track preferences. The algorithm calculates the Preference_episode for each placement and uses self-competition to determine the final selected placement. This process is crucial for optimizing controller placement in a network.

In Algorithm 2, (lines 1 to 2) is a procedure used within this Algorithm to update a buffer B. It takes as input the buffer B, the current placement $Placement_i$, and the $Preference_episode$ calculated during placement selection. This algorithm updates the buffer with the new preference value based on a buffer update factor α.

Algorithm 3 line 1 to 5 is another subroutine used within Algorithm 1. It calculates the self-competition reward for a placement based on a comparison between the preference value of the placement and the previous exponential moving average (EMA) value stored in the buffer B. It calculates the self-competition reward for a placement by comparing its preference value with the previous exponential moving average (EMA) stored in the buffer B. If the placement's preference is greater than or equal to the EMA, it returns 1; otherwise, it returns -1.

Algorithm 4 line 1 to 2 outlines the procedure for calculating the Preference_episode for a placement based on criteria_1 ($Cr1$) and criteria_2 ($Cr2$) with corresponding weights (w1 and w2). This procedure is a crucial step in Algorithm 1 for selecting controller placements. It calculates the Preference_episode for a placement by combining the weighted criteria w1 and w2. The weights w1 and w2 determine the importance of each criterion in the placement selection process.

5. Implementation and results

5.1. Implementation of self-competition for leader node selection in a cluster

To implement self-competition, where each node in the cluster competes with itself to become the leader, follow these steps: 1. Define the grid size to match the node cluster. 2. Each node will be represented by a unique position in the grid. 3. Initialize the game with all nodes active and no node as the leader. 4. Players can take actions to activate or deactivate nodes, simulating node preferences for minimal latency, minimal load, and minimal communication overhead. 5. A player can activate or deactivate a node, optimizing the node's specific criteria to become the leader. 6. The game ends when the number of active nodes reaches 1 (only one active node), and the player who achieves this final state is declared the winner. 7. For the reward function, we used the evaluation function (1) for each node p using the following formula: 8. A reward of 1 is assigned to the player who achieves the maximum value of Eval(p), indicating that they are the optimal leader node based on the criteria. Other players would receive a reward of 0. This encourages players to optimize their criteria and become the leader of the cluster. 9. Different importance levels are defined for each criterion to reflect their relative impact in the evaluation function. 10. At each step of the game, each node will perform actions to activate or deactivate nodes and calculate its own evaluation function using the specific criteria. The node will try to maximize its evaluation function to become the leader of the cluster. By adapting the game in this way, each node in the cluster engages in self-competition to become the leader by optimizing specific criteria. Each node uses its own evaluation function to guide its actions and seeks to maximize this evaluation in order to become the optimal leader node. To illustrate our approach, we have developed a game class called Game, which represents a node cluster environment. we chose the green cluster as shown in Fig. 7 which consists of 10 nodes, and the goal of the game is to determine the most performing leader nodes.

In our implementation, each node is represented by an array of three values. The first value indicates whether the node is active (1 for active, 0 for inactive), the second value represents the node's evaluation in terms of performance, and the third value is reserved for future use.

The game provides several functionalities. Firstly, it provides a *legalactions* method to obtain the available legal actions at a given moment. An action involves deactivating an active node. Then, the *applyaction* method allows applying a specific action by deactivating the corresponding node. The game uses an evaluation function to evaluate each node based on different criteria such as latency, load, and additional costs. The relative importance levels of these criteria are also taken into account. The overall evaluation of a node is calculated by weighting each criterion with its respective importance.

The game also allows updating the node evaluations at each step using the *updateevaluations* method. This method recalculates the evaluations of all nodes using the evaluation function.

Another important feature of the game is determining the leader nodes. The getleader method returns the indices of nodes with the maximum evaluations. The leader nodes are those whose evaluations are highest among all nodes in the cluster.

Each game step involves selecting an action, applying that action, updating the node evaluations, and determining rewards based on the leader nodes. A reward is assigned to each node based on its presence among the leader nodes.

Finally, the game also provides a reset method to reset the game state and a render method to display the current state of the node cluster along with the leader nodes.

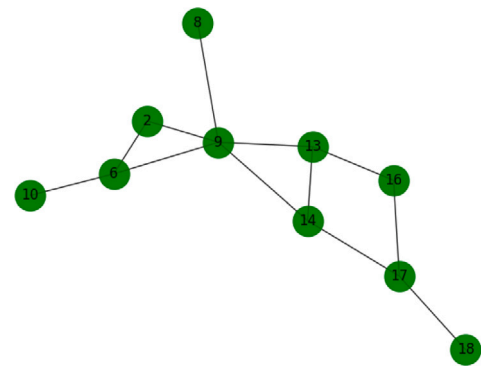

Fig. 7. Initial cluster.

5.2. Results and experiment

In this section, we present our experimental results using the MuZero implementation to solve the leader search problem in a cluster. We describe in detail the steps of our experiment, from data collection to the interpretation of the obtained results. We conducted experiments using the implemented prototype system to evaluate our approach. In the experiments, a PC using Intel Core i7 2.7 GHz, 16 GB memory and a Windows operating system is operated to run the prototype system. We have implemented a prototype of our approach by following the different steps described in Github link[1] and providing the developed source code.

5.2.1. Data collection

In this work, we used a random data in our experiment but future studies will include an evaluation of the suggested technique in four topologies, i.e., OS3E, USNet, Abilene, utilizing Mininet. We obtained the topologies from the topology zoo dataset. We used a graph Fig. 7 to represent a cluster for training MuZero. The graph consists of a certain number of nodes, each with specific attributes such as latency, load, and overhead. Firstly, we defined a suitable data structure to store the results of each game step. We opted for a list that allows us to store the information in an ordered and accessible manner.

At each game step, we recorded relevant information in our results data structure. This includes the current state of the game, i.e., the state of the nodes in the grid (activated or deactivated), the actions taken by each node at that specific step, the evaluation functions (Eval(p)) computed by each node, as well as the rewards assigned to each node (1 for the optimal leader node, 0 for the other nodes). In Initial state we have: Node 0: [1, 0, 0]

Node 1: [1, 0, 0]
Node 2: [1, 0, 0]
Node 3: [1, 0, 0]
Node 4: [1, 0, 0]
Node 5: [1, 0, 0]
Node 6: [1, 0, 0]
Node 7: [1, 0, 0]
Node 8: [1, 0, 0]
Node 9: [1, 0, 0]
Leader(s): []

Each line represents a node, and the numbers in square brackets ([1, 0, 0]) indicate node activation, and the following elements could be used for additional node information. Next, the selected action is applied to the cluster by calling the *applyaction* method, which deactivates the corresponding node. After that, the evaluations of all

[1] https://github.com/ouafeben/AOCP

Table 3
First action.

Node	Latency	Load	Overhead communication	Eval(p)	isleader
0	0.0716415227463526	0.36664155041357127	0.0011067337127277366	0.2739538017145041	No
1	0.8729080850577978	0.7711292329064804	0.0071960969140481	2.4367548416129026	No
2	0.1987174843440983	0.7779218661205913	0.741004323393509	3.977574186765808	No
3	0.306853006819897	0.7372603223125163	0.3683712009941429	2.886487254427358	No
4	0.600134064421151	0.36664155041357127	0.2837817368180384	2.184762375702409	No
5	0.4299051129543907	0.9558217131687736	0.177254299730926	2.873311438484716	No
6	0.6899703742529727	0.9049099965720787	0.35767546890058166	3.572816774098875	No
7	0.14162076608887364	0.819286549808912	0.8928332193788396	4.458693523843216	No
8	0.7536225353364361	0.9495724818839545	0.6717268177518089	4.667947952359771	Yes
9	0.6137667061844372	0.4205642239463119	0.5890868600218991	3.2221557341427585	No

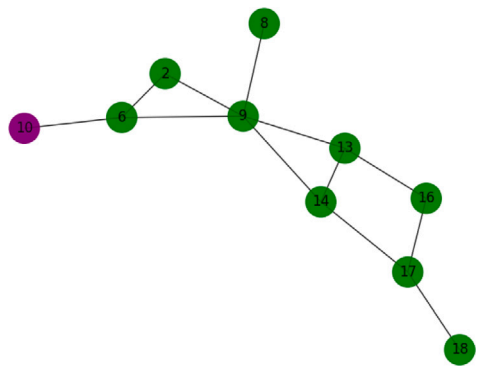

Fig. 8. Leader of cluster.

nodes are updated by calling the *updateevaluations* method. The reward is calculated based on the current leaders in the cluster. The list comprehension checks if each node index is in the list of leader nodes obtained from self.getleader(). If a node is a leader, its corresponding reward is set to 1; otherwise, it is set to 0. Finally, the rewards are summed up to get the total reward. step0: Table 3 shows the results after executing the action 0.the node 0 is deactivated, as indicated by the value [0, 0.2739538017145041, 0]. This means that the node is no longer active. The remaining nodes have their evaluations updated. Here are the updated values for the remaining nodes:

The leader node(s) are determined based on the evaluations. In this case, node 8 has the highest evaluation, so it is the leader. The list of leaders is [8]. -After executing the action 1, the node 1 is deactivated, as indicated by the value [0,3.9528935926059994, 0]. This means that the node is no longer active. The remaining nodes have their evaluations updated. Here are the updated values for the remaining nodes: Node 0: [0,2.501310984779907,0]

Node 2: [1,3.016771074211486,0]
Node 3: [1,1.2735947570403505,0]
Node 4: [1,2.2896238099069945,0]
Node 5: [1,1.6561848946213413,0]
Node 6: [1,3.696865374685697,0]
Node 7: [1,3.0331010587982763,0]
Node 8: [1,1.1231196304288802,0]
Node 9: [1,1.549048502618842,0]

The leader node(s) are determined based on the evaluations. In this case, node 1 was the leader before it was deactivated. Therefore, the new leader is determined, and in this case, it is node 6. The list of leaders is [1]. The tree depth is reported as 46, which indicates the depth of the game tree after the step. The root value for player 0 is 5.29, indicating the evaluation of the current state for player 0. The same process is repeated and at each time the node which has the max evaluation is the leader until the end of the Actions. The last action was:

[SelfPlay pid=26836] Played action: 9
(SelfPlay pid=26836) Cluster Nodes:
(SelfPlay pid=26836) Node 0: [0, 2.5818069718880134, 0]
(SelfPlay pid=26836) Node 1: [0, 2.385894472700738, 0]
(SelfPlay pid=26836) Node 2: [0, 2.3190047525852613, 0]
(SelfPlay pid=26836) Node 3: [0, 1.4989738276243836, 0]
(SelfPlay pid=26836) Node 4: [0, 4.182803797549335, 0]
(SelfPlay pid=26836) Node 5: [0, 4.09335500081362, 0]
(SelfPlay pid=26836) Node 6: [0, 2.398885767359116, 0]
(SelfPlay pid=26836) Node 7: [0, 3.84143333319191, 0]
(SelfPlay pid=26836) Node 8: [1, 2.0036617501638574, 0]
(SelfPlay pid=26836) Node 9: [0, 2.8828158848916403, 0]
(SelfPlay pid=26836) Leader(s): [4] as shows Fig. 8 .

By recording the results of each step for each node, we were able to analyze the complete gameplay and the individual performances of each node. At the end of the game, we obtained a comprehensive collection of results that recorded the actions, evaluation functions, and rewards of each node at each step. This data collection allowed us to evaluate the performances of the different nodes and analyze the strategies used by players to become the leader of the cluster. In addition to the basic information, we also added additional details such as the number of steps required to reach the final state with only one active node and the total elapsed time. These additional pieces of information offered us a more comprehensive overview of the self-competition. By adapting the data collection in this manner, we were able to analyze and evaluate the performances of the different nodes in the self-competition, providing valuable insights for our study.

5.2.2. MuZero training

We adapted the MuZero algorithm for our specific problem of leader search in a cluster. The training process of MuZero involves generating simulations using a search tree based on the possible actions in the graph. We used a deep neural network to estimate the values and policies in the game states. The training was conducted on a pre-generated training dataset derived from our cluster graphs.

LossRewardLoss: During training, we monitored the evolution of several evaluation metrics, including the loss calculated during the update of values and policies. We also recorded the LossRewardloss obtained at each training step to evaluate the overall performance of MuZero.

The “3.LossRewardloss” tag in the context of MuZero refers to a specific loss metric called “LossRewardloss”. This metric is used to evaluate the quality of reward predictions made by MuZero during its training.

In the context of MuZero, the reward is a measure of the quality of an action taken by the algorithm in a given state. The reward is often defined by the specific environment or problem to which MuZero is applied. For example, in a game, the reward can be the obtained score, victory or defeat, or any other performance criterion.

The “LossRewardloss” is a measure of the discrepancy between MuZero’s reward predictions and the actual rewards. It assesses how accurate MuZero’s reward predictions are. The goal of learning is to minimize this loss to improve MuZero’s ability to correctly estimate potential rewards for each action.

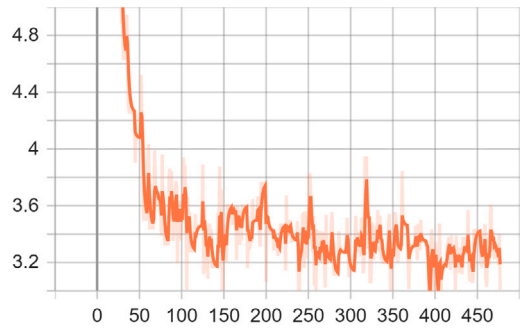

Fig. 9. Run-.-tag-3.LossRewardloss.

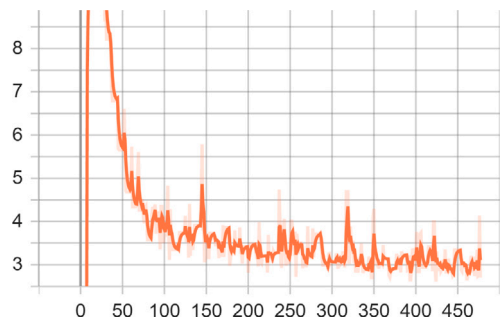

Fig. 10. Run-.-tag-3.LossValueloss.

In summary, "3.LossRewardloss" is an indicator of the quality of reward predictions made by MuZero during its training. A low value of this loss indicates accurate reward predictions and demonstrates that MuZero is capable of correctly estimating potential rewards.

Fig. 9 shows that the values of "LossRewardloss" decrease over the training iterations, indicating that our model is gradually adjusting and learning to optimize the provided rewards to approach the desired outcomes. A consistent decrease in this loss metric is generally desirable as it signifies that our model is improving and better adapting to our specific problem.

LossValueloss: The value of loss is a measure of the difference between the model's predictions and the expected values. Generally, a lower loss value indicates better performance of the model, as it means that the model's predictions are closer to the expected values.

In our context, Fig. 10 depicts the evolution of the loss value during the model training. in Fig. 11 We observe that the loss value is initially maximized at the beginning of training and gradually decreases until it reaches a value of 2.7. This indicates an improvement in the model's performance and predictions.

Learningrate: Fig. 11 shows that the learning rate initially increases until reaching a maximum value, and then exponentially decreases during training. This indicates the use of a technique called scheduled learning rate.

This approach aims to leverage the benefits of high learning rates in the early stages of training, such as fast convergence, while avoiding potential issues associated with high learning rates in the long term.

At the beginning of training, a high learning rate allows the model's weights to quickly explore a reasonable region of the search space. This can be beneficial as significant weight updates facilitate exploration of different areas in the weight space and can help the model escape local minima.

However, as training progresses, it is generally desirable to gradually reduce the learning rate. This allows for smaller and more precise weight updates, facilitating convergence towards a global minimum of the loss function.

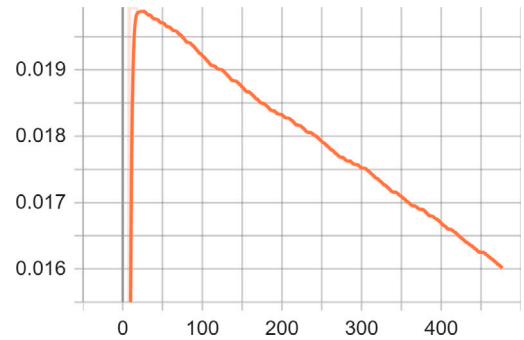

Fig. 11. LearningRate.

The exponential decrease of the learning rate is a commonly used approach to implement this strategy. For example, the learning rate could be defined as an exponential function of the number of training iterations or epochs. Over time, the learning rate will gradually decrease, enabling increasingly smaller weight updates.

This approach combines the advantages of high learning rates in the early stages of training with the stability and precision of lower learning rates as the model approaches convergence.

In summary, the use of a learning rate that initially increases and then exponentially decreases during training is a common strategy to achieve a trade-off between fast convergence and model stability.

To evaluate our technique in a large-scale network, we conducted experiments on a cluster of 72 nodes as shown on Fig. 12. the cluster was obtained after applying K-means algorithm on a network of 200 nodes. Here are the updated values for the remaining nodes:

(SelfPlay pid=10764) Node 12: [0, 1.9693689714061515, 0]
(SelfPlay pid=10764) Node 13: [0, 3.0487348204966516, 0]
(SelfPlay pid=10764) Node 14: [0, 3.887878416264744, 0]
(SelfPlay pid=10764) Node 15: [0, 4.589791078048341, 0]
(SelfPlay pid=10764) Node 16: [0, 4.311327749738352, 0]
(SelfPlay pid=10764) Node 17: [0, 3.5799221978974733, 0]
(SelfPlay pid=10764) Node 18: [0, 2.409944074002282, 0]
(SelfPlay pid=10764) Node 19: [0, 2.985122735386794, 0]
(SelfPlay pid=10764) Node 20: [0, 1.9818785684960798, 0]
(SelfPlay pid=10764) Node 21: [0, 4.741502160979575, 0]
(SelfPlay pid=10764) Node 22: [0, 3.308177326037949, 0]
(SelfPlay pid=10764) Node 23: [0, 3.9669411701343305, 0]
(SelfPlay pid=10764) Node 24: [0, 2.8792281947492775, 0]
(SelfPlay pid=10764) Node 25: [0, 4.070821246685206, 0]
(SelfPlay pid=10764) Node 26: [0, 3.49872396639772, 0]
(SelfPlay pid=10764) Node 27: [0, 1.6886882668411096, 0]
(SelfPlay pid=10764) Node 28: [0, 2.677419906902365, 0]
(SelfPlay pid=10764) Node 29: [0, 3.763402841239115, 0]
(SelfPlay pid=10764) Node 30: [0, 4.300251682498091, 0]
(SelfPlay pid=10764) Node 31: [0, 3.518054370006857, 0]
(SelfPlay pid=10764) Node 32: [0, 3.2534922714856593, 0]
(SelfPlay pid=10764) Node 33: [0, 4.14300473460151, 0]
(SelfPlay pid=10764) Node 34: [0, 3.2437446957049256, 0]
(SelfPlay pid=10764) Node 35: [0, 2.644153233467388, 0]
(SelfPlay pid=10764) Node 36: [0, 0.39212930548234937, 0]
(SelfPlay pid=10764) Node 37: [0, 2.952166728969001, 0]
(SelfPlay pid=10764) Node 38: [0, 3.1622491697595807, 0]
(SelfPlay pid=10764) Node 39: [0, 4.171849934893672, 0]
(SelfPlay pid=10764) Node 40: [0, 2.276168498644073, 0]
(SelfPlay pid=10764) Node 41: [0, 2.886673963176045, 0]
(SelfPlay pid=10764) Node 42: [0, 2.107446022432318, 0]
(SelfPlay pid=10764) Node 43: [1, 3.4036766046742524, 0]
(SelfPlay pid=10764) Node 44: [0, 2.8373876020885667, 0]
(SelfPlay pid=10764) Node 45: [0, 1.6951333970154576, 0]
(SelfPlay pid=10764) Node 46: [0, 4.389208297588453, 0]

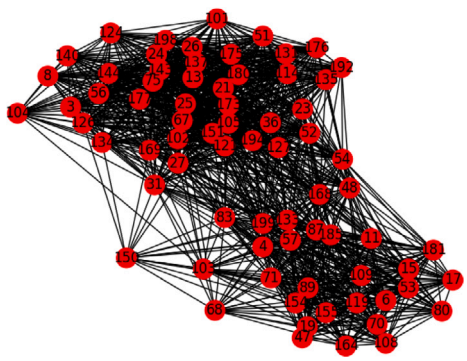

Fig. 12. Large cluster.

(SelfPlay pid=10764) Node 47: [0, 2.4141185126206386, 0]
(SelfPlay pid=10764) Node 48: [0, 2.376315419449555, 0]
(SelfPlay pid=10764) Node 49: [0, 2.4312605047679536, 0]
(SelfPlay pid=10764) Node 50: [0, 1.9956516735338634, 0]
(SelfPlay pid=10764) Node 51: [0, 3.4639004499399304, 0]
(SelfPlay pid=10764) Node 52: [0, 3.6539490081932184, 0]
(SelfPlay pid=10764) Node 53: [0, 1.030272407899216, 0]
(SelfPlay pid=10764) Node 54: [0, 3.0646481740878504, 0]
(SelfPlay pid=10764) Node 55: [0, 4.1310920074214295, 0]
(SelfPlay pid=10764) Node 56: [0, 2.4360148257347047, 0]
(SelfPlay pid=10764) Node 57: [0, 2.1717392922149443, 0]
(SelfPlay pid=10764) Node 58: [0, 0.813714785474129, 0]
(SelfPlay pid=10764) Node 59: [0, 4.503845318457003, 0]
(SelfPlay pid=10764) Node 60: [0, 2.3376387666431793, 0]
(SelfPlay pid=10764) Node 61: [0, 2.595867176501946, 0]
(SelfPlay pid=10764) Node 62: [0, 4.011671377019857, 0]
(SelfPlay pid=10764) Node 63: [0, 1.7697172279427034, 0]
(SelfPlay pid=10764) Node 64: [0, 4.343693391259612, 0]
(SelfPlay pid=10764) Node 65: [0, 4.1403014012289265, 0]
(SelfPlay pid=10764) Node 66: [0, 1.9925459575495728, 0]
(SelfPlay pid=10764) Node 67: [0, 3.2347262291479564, 0]
(SelfPlay pid=10764) Node 68: [0, 1.1053788638325646, 0]
(SelfPlay pid=10764) Node 69: [0, 2.784967383452708, 0]
(SelfPlay pid=10764) Node 70: [0, 2.934131878828839, 0]
(SelfPlay pid=10764) Node 71: [0, 3.101925865900653, 0]
(SelfPlay pid=10764) Leader(s): [9] as you can see the node chosen as controller is node 10(9+1)

We recorded the LossRewardloss obtained at each training step, the LossValueloss and the LearningRate to evaluate the overall performance of MuZero. Fig. 13, Fig. 14 and Fig. 15 show respectively the metrics recorded cited previously. I observed that the results obtained for loss reward, loss value, and the learning rate of MuZero were nearly the same. Despite the scale of the cluster, the evaluated metrics demonstrated remarkable consistency, highlighting the robustness and reliability of MuZero's performance at a large scale.

5.2.3. Results obtained

Our experimental results have shown that MuZero was able to efficiently find the leader in a cluster. Additionally, the time required to find the leader was on average less than 0.001 s in the small cluster and the large cluster, demonstrating the algorithm's effectiveness in complex search tasks. We also conducted graphical visualizations to illustrate MuZero's performance over time. The graphs show a progressive improvement in the accuracy of leader search as the training progressed.

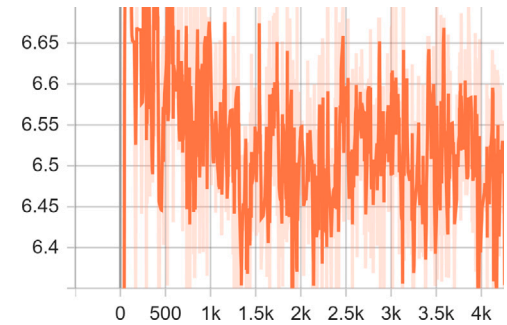

Fig. 13. LossRewardloss.

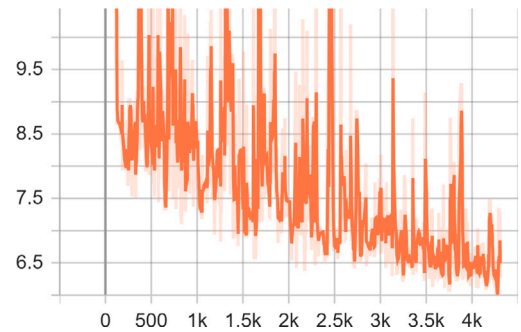

Fig. 14. Lossvalueloss.

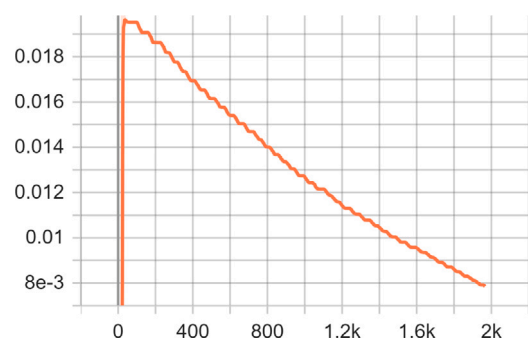

Fig. 15. Learning rate.

5.2.4. Analysis and interpretation of results

The analysis of our results highlights several significant observations. Firstly, MuZero's efficiency in leader search can be attributed to its ability to explore the different possible actions in the graph and learn adaptive strategies based on the game state. Furthermore, the use of a deep neural network allows MuZero to generalize the learned knowledge to new cluster graphs, providing it with high adaptability and generalization capabilities. By observing the loss curves during training, we observed a gradual decrease in loss over iterations, indicating that the MuZero model is converging towards more accurate estimations of values and policies in the game states. This convergence is an indicator of effective learning by the model. Analyzing the leader search times allows us to see that MuZero is capable of quickly finding a solution in most cases. However, in some cases where the cluster graph is highly complex, the search time may be slightly longer. This highlights the importance of balancing efficiency and accuracy of search in complex environments. We have identified three key strengths of using MuZero for leader selection in a cluster: Adaptability and Generalization: MuZero's ability to adapt to different environments or cluster configurations. Through training, MuZero

learns effective strategies and makes relevant leader selection decisions, even in complex or changing situations. Exploitation of Implicit Knowledge: The advantage of the training-based approach in terms of capturing implicit knowledge. MuZero is capable of learning patterns and strategies from self-competition data, enabling it to make more informed and contextually appropriate decisions in the cluster. Adaptation to Changes: MuZero's ability to adapt to dynamic changes in the cluster. Through continuous learning and self-competition, MuZero can adjust its strategies and leader selection in real-time, which is crucial in an environment where conditions can change rapidly. In conclusion, our experimental results demonstrate that the implementation of MuZero for leader selection in a cluster is promising. MuZero successfully finds the leader in complex networks, with high success rates and reasonable search times. These results pave the way for the use of MuZero in other leader search and navigation problems in complex environments.

6. Conclusion and further research

In conclusion, this paper presents a novel and promising solution for controller placement in SDN environments. The implementation of the MuZero reinforcement learning algorithm coupled with self-competition demonstrates its effectiveness in optimizing the placement of controllers in large networks.

By leveraging the capabilities of MuZero, our intelligent system surpasses human-level performance and achieves superhuman capabilities in complex domains. The integration of a learned model with tree search enables our system to find optimal controller placements with high success rates, even in intricate network structures.

The significance of our solution lies in its ability to enhance network control and management in SDN. By strategically placing controllers, our system improves network flexibility and enables more efficient and effective network operations. This has important implications for achieving improved network performance, scalability, and overall network management.

The experimental results validate the potential of our approach, showcasing its ability to successfully find the leader in complex networks. The reasonable search times further highlight the practicality and efficiency of our solution in real-world scenarios.

Moving forward, our solution opens up new avenues for research and practical applications in controller placement optimization. Further enhancements and customizations can be explored to address specific network environments and challenges, ensuring the scalability and adaptability of our solution.

Overall, our proposed solution offers a promising and intelligent approach to controller placement in SDN, contributing to the advancement of network control and management in large-scale networks.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Potential competing interests with the following people: Saïd Jabbour, Simon Collart Dutilleul, Zakaryae Boudi, Rahma Ben Ayed, Mohamed Haloua, Akram Idani, Yassine Ouhammou, Mouna Rekik, Yves Ledru, Mohamed Toub, Sébasten Limet, Sophie Robert, Slim Kallel, Riadh Ben Halima, Yacine Sam, Walid Gaaloul, Mohamed Sellami, Souha Boubaker, Nasredine Cheniki, Philippe Bon, Racem Bounacha, Khalifa Toumi, Salima Benbernou, Jean-Luc Rafy, Abdessamad Jarrar, Asma Lahbib, Khouloud Boukadi, Anis Laouiti, Khaoula Bouba, Ladjel Bellatreche, Steven Martin, Ahmed Awad, Emna Guermazi, Youssef Balouki, Mouad Ben Mamoun, Fouad Benamrane, Pierre Vincent, Karim Sbata, Mounir Azizi, Othmane Blial, Adnane Mzibri

References

Ahmad, Suhail, Mir, Ajaz Hussain, 2021. Scalability, consistency, reliability and security in SDN controllers: a survey of diverse SDN controllers. J. Netw. Syst. Manage. 29, 1–59.

Ahmadi, Vahid, Khorramizadeh, Mostafa, 2018. An adaptive heuristic for multi-objective controller placement in software-defined networks. Comput. Electr. Eng. 66, 204–228.

Altman, Eitan, 1999. Constrained Markov Decision Processes, Vol. 7. CRC Press.

Amin, Azam, Jahanshahi, Mohsen, Meybodi, Mohammadreza, 2021. A new clustering approach for efficient placement of controllers in SDN using firefly algorithm. Int. J. Smart Electr. Eng. 10 (03), 169–175.

Aravind, P., Varma, G.P. Saradhi, Reddy, P.V.G.D. Prasad, 2022. Simulated annealing based optimal controller placement in software defined networks with capacity constraint and failure awareness. J. King Saud Univ. Comput. Inf. Sci. 34 (8), 5721–5733.

Arulkumaran, Kai, Deisenroth, Marc Peter, Brundage, Miles, Bharath, Anil Anthony, 2017. A brief survey of deep reinforcement learning. arXiv preprint arXiv:1708.05866.

Ateya, Abdelhamied A., Muthanna, Ammar, Vybornova, Anastasia, Algarni, Abeer D., Abuarqoub, Abdelrahman, Koucheryavy, Y., Koucheryavy, Andrey, 2019. Chaotic salp swarm algorithm for SDN multi-controller networks. Eng. Sci. Technol. Int. J. 22 (4), 1001–1012.

Auroux, Sébastien, Karl, Holger, 2014. Flow processing-aware controller placement in wireless DenseNets. In: 2014 IEEE 25th Annual International Symposium on Personal, Indoor, and Mobile Radio Communication. PIMRC, IEEE, pp. 1294–1299.

Benzekki, Kamal, El Fergougui, Abdeslam, Elbelrhiti Elalaoui, Abdelbaki, 2016. Software-defined networking (SDN): a survey. Secur. Commun. Netw. 9 (18), 5803–5833.

Bouzidi, E.L. Hocine, Outtagarts, Abdelkader, Langar, Rami, Boutaba, Raouf, 2022. Dynamic clustering of software defined network switches and controller placement using deep reinforcement learning. Comput. Netw. 207, 108852.

Bowling, Michael, Veloso, Manuela, 2000. An Analysis of Stochastic Game Theory for Multiagent Reinforcement Learning. Citeseer.

Browne, Cameron B., Powley, Edward, Whitehouse, Daniel, Lucas, Simon M, Cowling, Peter I., Rohlfshagen, Philipp, Tavener, Stephen, Perez, Diego, Samothrakis, Spyridon, Colton, Simon, 2012. A survey of Monte Carlo tree search methods. IEEE Trans. Comput. Intell. AI Games 4 (1), 1–43.

Coulom, Rémi, 2006. Efficient selectivity and backup operators in Monte-Carlo tree search. In: International Conference on Computers and Games. Springer, pp. 72–83.

Farhady, Hamid, Lee, HyunYong, Nakao, Akihiro, 2015. Software-defined networking: A survey. Comput. Netw. 81, 79–95.

Hakiri, Akram, Gokhale, Aniruddha, Berthou, Pascal, Schmidt, Douglas C, Gayraud, Thierry, 2014. Software-defined networking: Challenges and research opportunities for future internet. Comput. Netw. 75, 453–471.

He, Mu, Kalmbach, Patrick, Blenk, Andreas, Kellerer, Wolfgang, Schmid, Stefan, 2017. Algorithm-data driven optimization of adaptive communication networks. In: 2017 IEEE 25th International Conference on Network Protocols. ICNP, IEEE, pp. 1–6.

Heller, Brandon, Sherwood, Rob, McKeown, Nick, 2012. The controller placement problem. ACM SIGCOMM Comput. Commun. Rev. 42 (4), 473–478.

Hu, Ying, Luo, Tao, Beaulieu, Norman C., Deng, Chunxue, 2016. The energy-aware controller placement problem in software defined networks. IEEE Commun. Lett. 21 (4), 741–744.

Huang, Victoria, Chen, Gang, Zhang, Peng, Li, Hao, Hu, Chengchen, Pan, Tian, Fu, Qiang, 2020. A scalable approach to SDN control plane management: High utilization comes with low latency. IEEE Trans. Netw. Serv. Manag. 17 (2), 682–695.

Ishigaki, Genya, Gour, Riti, Yousefpour, Ashkan, Shinomiya, Norihiko, Jue, Jason P, 2017. Cluster leader election problem for distributed controller placement in sdn. In: GLOBECOM 2017-2017 IEEE Global Communications Conference. IEEE, pp. 1–6.

Jalili, Ahmad, Keshtgari, Manijeh, Akbari, Reza, Javidan, Reza, 2019. Multi criteria analysis of controller placement problem in software defined networks. Comput. Commun. 133, 115–128.

Kaelbling, Leslie Pack, Littman, Michael L., Moore, Andrew W., 1996. Reinforcement learning: A survey. J. Artif. Intell. Res. 4, 237–285.

Keshari, Surendra Kumar, Kansal, Vineet, Kumar, Sumit, 2021. An intelligent way for optimal controller placements in software-defined–IoT networks for smart cities. Comput. Ind. Eng. 162, 107667.

Killi, Bala Prakasa Rao, Rao, Seela Veerabhadreswara, 2018. Link failure aware capacitated controller placement in software defined networks. In: 2018 International Conference on Information Networking. ICOIN, IEEE, pp. 292–297.

Killi, Bala Prakasa Rao, Rao, Seela Veerabhadreswara, 2019. Towards improving resilience of controller placement with minimum backup capacity in software defined networks. Comput. Netw. 149, 102–114.

Ksentini, Adlen, Bagaa, Miloud, Taleb, Tarik, 2016. On using SDN in 5G: The controller placement problem. In: 2016 IEEE Global Communications Conference. GLOBECOM, IEEE, pp. 1–6.

Lange, Stanislav, Gebert, Steffen, Spoerhase, Joachim, Rygielski, Piotr, Zinner, Thomas, Kounev, Samuel, Tran-Gia, Phuoc, 2015a. Specialized heuristics for the controller placement problem in large scale SDN networks. In: 2015 27th International Teletraffic Congress. IEEE, pp. 210–218.

Lange, Stanislav, Gebert, Steffen, Zinner, Thomas, Tran-Gia, Phuoc, Hock, David, Jarschel, Michael, Hoffmann, Marco, 2015b. Heuristic approaches to the controller placement problem in large scale SDN networks. IEEE Trans. Netw. Serv. Manag. 12 (1), 4–17.

Li, Chunlin, Jiang, Kun, Luo, Youlong, 2022. Dynamic placement of multiple controllers based on SDN and allocation of computational resources based on heuristic ant colony algorithm. Knowl.-Based Syst. 241, 108330.

Liao, Jianxin, Sun, Haifeng, Wang, Jingyu, Qi, Qi, Li, Kai, Li, Tonghong, 2017. Density cluster based approach for controller placement problem in large-scale software defined networkings. Comput. Netw. 112, 24–35.

Long, Qingyue, Chen, Yanliang, Zhang, Haijun, Lei, Xianfu, 2019. Software defined 5G and 6G networks: A survey. Mobile Netw. Appl. 1–21.

Mohanty, Sagarika, Kanodia, Khushboo, Sahoo, Bibhudatta, Kurroliya, Kuldeep, 2020. A simulated annealing strategy for reliable controller placement in software defined networks. In: 2020 7th International Conference on Signal Processing and Integrated Networks. SPIN, IEEE, pp. 844–849.

Mostafaei, Habib, Menth, Michael, Obaidat, Mohammad S., 2018. A learning automaton-based controller placement algorithm for software-defined networks. In: 2018 IEEE Global Communications Conference. GLOBECOM, IEEE, pp. 1–6.

Müller, Lucas F., Oliveira, Rodrigo R., Luizelli, Marcelo C., Gaspary, Luciano P., Barcellos, Marinho P., 2014. Survivor: An enhanced controller placement strategy for improving SDN survivability. In: 2014 IEEE Global Communications Conference. IEEE, pp. 1909–1915.

Nowé, Ann, Vrancx, Peter, De Hauwere, Yann-Michaël, 2012. Game theory and multi-agent reinforcement learning. Reinf. Learn. State-of-the-Art 441–470.

Pujolle, Guy, 2015. Wi-Fi and 5G. Softw. Netw. Virtual. SDN 5G Secur. 1, 137–187.

Qin, Qiaofeng, Poularakis, Konstantinos, Iosifidis, George, Tassiulas, Leandros, 2018. SDN controller placement at the edge: Optimizing delay and overheads. In: IEEE INFOCOM 2018-IEEE Conference on Computer Communications. IEEE, pp. 684–692.

Sahoo, Kshira Sagar, Puthal, Deepak, Obaidat, Mohammad S., Sarkar, Anamay, Mishra, Sambit Kumar, Sahoo, Bibhudatta, 2018. On the placement of controllers in software-defined-WAN using meta-heuristic approach. J. Syst. Softw. 145, 180–194.

Sahoo, Kshira Sagar, Sahoo, Bibhudatta, Dash, Ratnakar, Tiwary, Mayank, 2016. Solving multi-controller placement problem in software defined network. In: 2016 International Conference on Information Technology. ICIT, IEEE, pp. 188–192.

Sallahi, Afrim, St-Hilaire, Marc, 2014. Optimal model for the controller placement problem in software defined networks. IEEE Commun. Lett. 19 (1), 30–33.

Sarmiento, David Espinel, Lebre, Adrien, Nussbaum, Lucas, Chari, Abdelhadi, 2021. Decentralized SDN control plane for a distributed cloud-edge infrastructure: A survey. IEEE Commun. Surv. Tutor. 23 (1), 256–281.

Savas, S. Sedef, Tornatore, Massimo, Dikbiyik, Ferhat, Yayimli, Aysegul, Martel, Charles U, Mukherjee, Biswanath, 2018. RASCAR: Recovery-aware switch-controller assignment and routing in SDN. IEEE Trans. Netw. Serv. Manag. 15 (4), 1222–1234.

Schrittwieser, Julian, Antonoglou, Ioannis, Hubert, Thomas, Simonyan, Karen, Sifre, Laurent, Schmitt, Simon, Guez, Arthur, Lockhart, Edward, Hassabis, Demis, Graepel, Thore, et al., 2020. Mastering atari, go, chess and shogi by planning with a learned model. Nature 588 (7839), 604–609.

Silver, David, Huang, Aja, Maddison, Chris J., Guez, Arthur, Sifre, Laurent, Van Den Driessche, George, Schrittwieser, Julian, Antonoglou, Ioannis, Panneershelvam, Veda, Lanctot, Marc, et al., 2016. Mastering the game of Go with deep neural networks and tree search. Nature 529 (7587), 484–489.

Singh, Sanjeev, Jha, Rakesh Kumar, 2017. A survey on software defined networking: Architecture for next generation network. J. Netw. Syst. Manage. 25, 321–374.

Singh, Ashutosh Kumar, Maurya, Saurabh, Srivastava, Shashank, 2020. Varna-based optimization: a novel method for capacitated controller placement problem in SDN. Front. Comput. Sci. 14, 1–26.

Sood, Keshav, Yu, Shui, Xiang, Yong, Peng, Sancheng, 2016. Control layer resource management in SDN-IoT networks using multi-objective constraint. In: 2016 IEEE 11th Conference on Industrial Electronics and Applications. ICIEA, IEEE, pp. 71–76.

Sridharan, Vignesh, Gurusamy, Mohan, Truong-Huu, Tram, 2017. On multiple controller mapping in software defined networks with resilience constraints. IEEE Commun. Lett. 21 (8), 1763–1766.

St John, M.M., 2020. World models and predictive methods in deep reinforcement learning: A survey.

Tadros, Catherine Nayer, Rizk, Mohamed RM, Mokhtar, Bassem Mahmoud, 2020. Software defined network-based management for enhanced 5G network services. IEEE Access 8, 53997–54008.

Tanha, Maryam, Sajjadi, Dawood, Ruby, Rukhsana, Pan, Jianping, 2018. Capacity-aware and delay-guaranteed resilient controller placement for software-defined WANs. IEEE Trans. Netw. Serv. Manag. 15 (3), 991–1005.

Thalapala, Vidya Sagar, Guravaiah, Koppala, 2022. Fcmcp: Fuzzy c-means for controller placement in software defined networking. Procedia Comput. Sci. 201, 109–116.

Thalapala, Vidya Sagar, Mohan, Anandhu, Guravaiah, Koppala, 2022. WOACCPP: Wisdom of artificial crowds for controller placement problem with latency and reliability in SDN-WAN.

Torkamani-Azar, Sahand, Jahanshahi, Mohsen, 2020. A new GSO based method for SDN controller placement. Comput. Commun. 163, 91–108.

Tuyls, Karl, Perolat, Julien, Lanctot, Marc, Leibo, Joel Z., Graepel, Thore, 2018. A generalised method for empirical game theoretic analysis. arXiv preprint arXiv:1803.06376.

Vizarreta, Petra, Machuca, Carmen Mas, Kellerer, Wolfgang, 2016. Controller placement strategies for a resilient SDN control plane. In: 2016 8th International Workshop on Resilient Networks Design and Modeling. RNDM, IEEE, pp. 253–259.

Wang, Guodong, Zhao, Yanxiao, Huang, Jun, Wu, Yulei, 2017. An effective approach to controller placement in software defined wide area networks. IEEE Trans. Netw. Serv. Manag. 15 (1), 344–355.

Wu, Shuai, Chen, Xiaoqian, Yang, Lei, Fan, Chengguang, Zhao, Yong, 2018. Dynamic and static controller placement in software-defined satellite networking. Acta Astronaut. 152, 49–58.

Xia, Wenfeng, Wen, Yonggang, Foh, Chuan Heng, Niyato, Dusit, Xie, Haiyong, 2014. A survey on software-defined networking. IEEE Commun. Surv. Tutor. 17 (1), 27–51.

Yuan, Tingting, Huang, Xiaohong, Ma, Maode, Yuan, Jie, 2018. Balance-based SDN controller placement and assignment with minimum weight matching. In: 2018 IEEE International Conference on Communications. ICC, IEEE, pp. 1–6.

Zhang, Bang, Wang, Xingwei, Huang, Min, 2018. Multi-objective optimization controller placement problem in internet-oriented software defined network. Comput. Commun. 123, 24–35.